

Voici le résumé exhaustif du projet **K-Guard AI**, structuré autour de **ce qu'est le projet**, de sa **gestion technique**, et des **raisons stratégiques et professionnelles** qui motivent sa création.

1. Fiche d'Identité du Projet : K-Guard AI

- **Nature** : Microservice indépendant d'assistance intelligente pour la cybersécurité (AIOps / DevSecOps).
- **Technologies clés** : Java (Spring Boot / Spring AI), Kubernetes, LLM local open-source (via Ollama/vLLM), Proxy Guardrail (ex: LLM Guard / NeMo), Stack ELK.
- **Architecture** : Découplée du cœur de la plateforme K-Guard (architecture microservices), hébergée et orchestrée à 100 % au sein du cluster Kubernetes.

2. Description Fonctionnelle & Rôle de l'Application

K-Guard AI agit comme un **intermédiaire intelligent et souverain** entre les outils de détection de sécurité (Falco, Wazuh) et les équipes opérationnelles.

Le flux de fonctionnement :

1. **Reception de l'événement** : Lorsqu'une alerte critique ou un comportement anormal est détecté dans le cluster (ex: un shell interactif `bash` ouvert dans un pod par `root`), K-Guard transmet l'alerte brute à **K-Guard AI** via un webhook ou une API REST.
2. **Filtrage et Sécurisation (Guardrail)** : L'application Java nettoie et anonymise le log brut. Elle s'assure qu'aucun secret (mots de passe, tokens, clés d'API, données personnelles) ne soit transmis au modèle de langage.
3. **Analyse par LLM Local** : Le texte nettoyé est envoyé au Pod LLM local (ex: *Mistral-7B* ou *Llama-3* tournant sur Ollama/vLLM dans K8s). Le modèle synthétise l'événement.
4. **Enrichissement et Restitution** : L'application traduit la ligne de log technique complexe en un résumé clair en français comprenant :
 1. **L'explication de l'incident** (ce qui s'est passé).
 2. **L'évaluation du risque** (l'impact potentiel).
 3. **La recommandation d'action** (la procédure de remédiation conseillée).
5. **Observabilité** : L'analyse enrichie est renvoyée vers **ELK** pour être affichée dans un tableau de bord lisible et traçable.

3. Pourquoi tu veux faire ce projet ? (Les Motivations)

Tes motivations s'articulent autour de **quatre piliers majeurs** :

A. Résoudre un vrai problème opérationnel (La fatigue des alertes)

En SOC et en DevSecOps, la lecture de milliers de logs JSON bruts est chronophage et génère de la fatigue décisionnelle. K-Guard AI permet de contextualiser immédiatement la menace et d'accélérer la prise de décision lors d'un incident.

B. Pratiquer le LLMOps et la sécurité des IA de manière concrète

Plutôt que d'utiliser une IA externe (comme OpenAI) qui poserait des problèmes de confidentialité des données, tu mets en pratique la culture LLMOps :

- Déploiement et orchestration d'un modèle open-source sur Kubernetes.
- Maîtrise des risques de fuite de données (*Data Leakage*) et d'injection de consignes (*Prompt Injection*).
- Mise en place de proxys de sécurité (*Guardrails*) dédiés au langage naturel.

C. Valider une architecture microservices robuste avec Java / Spring Boot

Créer une application Java autonome permet de :

- Ne pas surcharger l'application K-Guard principale.
- Exploiter l'écosystème **Spring AI**, référence du marché entreprise pour l'intégration de LLM et de pipelines RAG.
- Démontrer ta capacité à concevoir des architectures distribuées et modulaires.

D. Consolider et valoriser un profil DevSecOps / Infra d'exception

Ce projet constitue une vitrine technique complète pour ta recherche d'alternance et tes échanges professionnels (notamment dans des secteurs exigeants comme la finance ou la banque) :

- **Posture humble et opérationnelle** : Tu pars de la base solide (Administration Système, Réseau, Kubernetes, Falco/ELK/Wazuh) pour construire progressivement une brique d'IA appliquée.
- **Différenciation sur le marché** : Tu associes la sécurité d'infrastructure critique (K8s / DevSecOps) à la maîtrise du développement backend (Java) et à la sécurisation des usages de l'IA (LLMOps).